

Ejercicio: Programa Multiarchivo en C++

Descripcion del ejercicio

Este ejercicio demuestra como organizar un proyecto en C++ utilizando **multiples archivos fuente**. El programa incluye:

- `main.cpp` : Punto de entrada del programa
 - `util.h / util.cpp` : Funciones utilitarias en un namespace (`saludar` , `sumar`)
 - `persona.h / persona.cpp` : Clase `Persona` con encapsulamiento
-

Requisitos previos

Compilador C++ (g++)

Linux (Ubuntu/Debian):

```
sudo apt update
sudo apt install build-essential
```

Alternativamente, puedes usar [MinGW](#) o [MSYS2](#).

CMake (opcional, si usas CMake en lugar de Make)

Linux:

```
sudo apt install cmake
```

macOS:

```
brew install cmake
```

Windows:

Descarga el instalador desde cmake.org

Como compilar y ejecutar

Opcion 1: Usando Make (recomendado)

Desde la carpeta `BibliotecasCpp` , ejecuta:

Compilar:

```
make
```

Ejecutar:

```
make run
```

Limpiar archivos compilados:

```
make clean
```

Opcion 2: Usando CMake

Desde la carpeta `BibliotecasCpp` , ejecuta:

Crear directorio de build y compilar:

```
mkdir build  
cd build  
cmake ..  
make
```

Ejecutar:

```
./programa
```

Opcion 3: Compilacion manual con g++

Si quieres entender que pasa detras de camara:

```
g++ main.cpp util.cpp persona.cpp -o programa  
./programa
```

Salida esperada

Al ejecutar el programa deberias ver:

```
Hola, Carlos!  
Suma: 8  
Nombre: Ana, Edad: 25
```

Estructura del proyecto

```
BibliotecasCpp/  
├─ main.cpp          # Funcion main  
├─ util.h            # Declaraciones de funciones utilitarias  
├─ util.cpp          # Implementacion de funciones utilitarias  
├─ persona.h         # Declaracion de la clase Persona  
├─ persona.cpp       # Implementacion de la clase Persona  
├─ Makefile          # Reglas para compilar con make  
└─ CMakeLists.txt    # Configuracion para compilar con cmake
```

Notas

- El archivo `persona.h` usa **include guards** (`#ifndef PERSONA_H`) para evitar inclusion multiple
- Las funciones utilitarias estan dentro del **namespace** `utils`
- La clase `Persona` usa **encapsulamiento** con atributos privados y metodos publicos